

Team Control Number
IMMC23061054

Problem Chosen

E

2023

IMMC

Summary Sheet

Biodiversity is essential for the processes that support all life on Earth, including humans. Along with more and more biological species have been detected and studied, some aspects of biodiversity are instinctively widely valued by people, which is the task of distinguishing homogeneous animal species that are difficult to categorize by their exterior appearance.

Showing our interest in this issue and tending to provide practical solutions, we offer unique insight through the construction of mathematical models. Specifically, our team builds three separate models based on the theory of Support Vector Machine (SVM), Adaptive Boosting Algorithm (Adaboost), and Random Forest, which aims for differentiating 8 lizard species and sex belonging to the genus *Darevskia* based on their measured data.

Based on the solid theorem of SVM, we construct models that are able to tackle all 5 tasks. Using FPNr, we identify the criterion that sets lizard species 5 apart from other species with an accuracy of **99.82%**(Task 1). The best pair of predicting variables are also explored, which reveals that the best accuracy of **95.74%** occurs when differentiating species 5 based on **FPNr** and **HFL** (Task 2). Besides, the lizards' sex is also predicted with an accuracy of **81.43%** (Task 3). The task of distinguishing lizards from certain subgroups is likewise completed with excellent accuracy of **84.56%**(numbers 6 and 7), **84.42%** (numbers 1 and 2), and **89%** (numbers 3, 4, and 5). (Task 4). In the end, integrating all the models mentioned above and combining the theorem of SVM, we discover a criterion with the ability to forecast the species and sex of lizards over the entire population, which possesses a precision of **72.34%** (Task 5).

For the outcomes that could be improved, we build 3 alternative models to achieve higher accuracy. To be more precise, as for task 2 (95.74% accuracy), we improve the result by constructing a model based on the theoretical underpinning of Adaboost. It performs admirably and shows a **99.82%** accuracy. For task 3, the approach of Random Forest has been introduced, and with the elimination of noises, the accuracy has been successfully improved from 81.43% to **88.23%** (originally 81.43% accuracy). For task 5, by combining Random Forest Classifier and SVM, the final accuracy is improved to **79.47%** (originally 72.97%).

As for our strength, we comprise our innovations beyond basic theoretical models. To illustrate, SVM is originally designed to support binary classification, but we utilize it to accomplish all the given tasks including multi-class classification, which demonstrates a solid theoretical foundation and high robustness. Besides, we have introduced highly targeted models for specific problems and successfully improved the accuracy. Besides, we encompass sensitivity analysis to ensure the explanation is as comprehensive as possible. Meanwhile, we also improve the integration model by optimizing each part of it, and the final integration was remarkable.

Considering all, our future expectation focus on optimizing the model by gathering more precise data about lizards and using a larger dataset for training and testing.

Keywords: Classification; Lizard; SMV; Adaboost; Random Forest

Contents

1	Introduction	1
1.1	Background	1
1.2	Problem Restatement	1
2	Assumptions and Justification	1
3	Variables	2
4	Support Vector Machine (SVM)	2
4.1	K-Fold Cross-Validation Method	2
4.2	Basic Theorem for SVM	3
4.3	Kernel Function	3
4.4	Distinguishing Species # 5 Using FPNr (Task 1)	5
4.5	The Best Pair of Predicting Variables for Species # 5 (Task 2)	5
4.6	Distinguishing Sex (Task 3)	6
4.7	Distinguishing Subgroup Species (Task 4)	7
4.7.1	Species # 6 and # 7 (4a)	7
4.7.2	Species # 1 and # 2 (4b)	7
4.7.3	Species # 3, # 4, and # 5 (4c)	8
4.8	Integreted Model- Predicting Species in the Entire Lizard Population (Task 5)	8
5	Alternative Models	9
5.1	Adaptive Boosting Algorithm (Adaboost)	9
5.1.1	Basic Theorems	9
5.1.2	Construction of Adaboost Algorithm	10
5.1.3	Task 2 Implementation	11
5.2	Random Forest	11
5.2.1	Basic Theorems	11
5.2.2	Decision Trees	11
5.2.3	Construction of Random Forest	12
5.2.4	Task 3 Implementation	12
5.2.5	Improved Random Forest Model	15

5.3	Improved Integrated Model	16
5.3.1	Basic Theroem	16
5.3.2	Task 5 Implementation	16
6	Sensitivity Analysis	17
6.1	SMV	17
6.2	Adaboost	17
6.3	Random Forest	18
7	Strengths and Weaknesses	18
7.1	Strengths	18
7.2	Weaknesses	19
8	Conclusion and Future Work	19
	Appendices	20
	Appendix A Task 4c. SVM Multiclass Classification	20
	Appendix B Task 2. Adaboost Algorithm	23
	Appendix C Task 3. SVM Classifier	24
	Appendix D Task 3. Random Forest	26

1 Introduction

1.1 Background

As the discipline of biology has progressed over the last few decades, more and more new species have been discovered. However, there are numerous homogeneous animals in our world. These animals have similar appearances and are indistinguishable without extensive data comparison and analysis. Major inaccuracies could be produced if the researchers do not deal with the data thoroughly.

A good example is the lizards belonging to *Darevskia*, a genus of wall lizards of the family Lacertidae[1]. There is no easy way to tell the lizards apart, despite the fact that females are often paler in color than males. However, this is always insufficient: Color differences are not always correct, and various people perceive colors differently, which can lead to inaccuracy.[2] They are so hard to be differentiated unless measuring the dimensions of their body.[3] To avoid extinction, aid reproduction, and boost the scientific research of such lizards, it is necessary for scientists to identify them correctly. Hence, it is very important to establish a mathematical model that helps scientists to numerically differentiate the closely related lizards.

1.2 Problem Restatement

We will answer the following prompts based on elaborated mathematical models:

- **Question 1:** Distinguish lizard species #5 from other species using the right side femoral pore number(FPNr).
- **Question 2:** Distinguish species #5 from other species using a pair of best criteria.
- **Question 3:** Predict the lizards' sex with the possibly best accuracy.
- **Question 4:** Distinguish subspecies that live adjacently.
- **Question 5:** Distinguish lizards' species and sex from the entire lizard population.

Specifically, we will construct robust mathematical models that enable us to tackle all five questions. We will also construct other mathematical models targeted at specific questions to achieve optimization.

2 Assumptions and Justification

To simplify the problem, we make the following basic assumptions, each of which is properly justified.

- **Assumption 1:** Data possess characteristics that can be classified.
- **Justification 1:** We expect the measured lizard data to possess unique attributes that can be classified, which is the foundation of the usage of classifier models.
- **Assumption 2:** No extreme anomalies in the given morphometric and pholidosis dataset.
- **Justification 2:** We expect there are no significant exceptions in the given dataset that could dramatically interfere with the manipulation of the model.

- **Assumption 3 :** Certain sites are inhabited only by species belonging to the same subgroup.
- **Justification 3 :** For task 4, we will expect all the sites mentioned to be inhabited only by species within the same subgroup. Species belonging to other subgroups should be totally absent.
- **Assumption 4 :** There is no correlation between lizards' sex and species.
- **Justification 4 :** We found that in each species, the number of lizards with different sex was basically the same and the number of species included in each sex was also relatively even, so we did not think they were correlated.

3 Variables

Table 3.1: Variables Table	
Variables	Description
E_t	Averaged total error
T	Training data set
D	Weight Matrix
a_t	Final wea classifier
IG	Information Gain
H_X	Information entropy
P	Probability
$H_{X Y}$	Conditional entropy

4 Support Vector Machine (SVM)

4.1 K-Fold Cross-Validation Method

Given the limited lizard sample, K-fold cross-validation is the method we use to yield less biased evaluations. We perform k-fold cross-validation as shown below:

1. We divided the dataset into k subsamples.
2. We first deem the 1st fold as the testing data and the rest $k - 1$ folds as the training data. We are able to get a percentage error E_1 .
3. After that we use the 2nd fold as the testing data and the rest as the training data. We are also able to get a percentage error E_2 .
4. The rest can be done in the same manner, we can get the percentage error from the first iteration (E_1) to the last (E_k). After k iterations, we can get the total error rate E_t by averaging all those k individual percentage errors (Equation 4.1). We will also average the results to achieve the overall mathematical model.

$$E_t = \frac{1}{k} \sum_{i=1}^k E_i \quad (4.1)$$

We will perform 8-fold cross-validation in all scenarios ($k=8$) because of the relatively small sample size. Besides, we choose $k=8$ due to the fact that there are 8 species present, and most of the classifications are based on biological species.

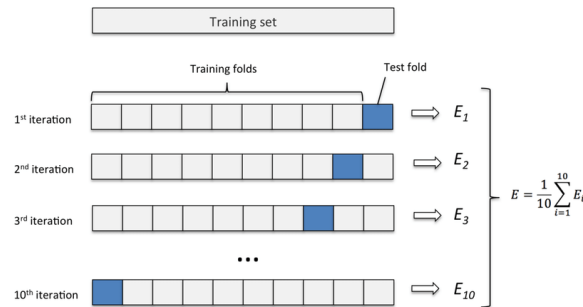


Figure 4.1: Illustration of a Typical 10-Fold Cross-Validation($k=10$) [4]

4.2 Basic Theorem for SVM

Support vector machine (SVM) can be either used as a regressive model or a distributive model. It separates objects belonging to different classes by finding the optimal hyperplane. Usually, we classify the problems that SVM can solve into the following three types:

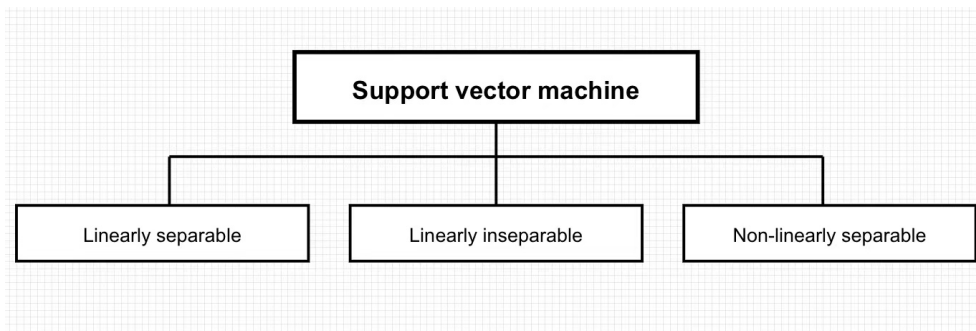


Figure 4.2: Classification of SVM

First, we can assume that our samples are linearly separable, then we are able to find a hyperplane (Equation 4.2):

$$w^T x + b = 0 \quad (4.2)$$

Where $w \in \mathbb{R}^{1 \times n}$ is the normal vector of the hyperplane, $x \in \mathbb{R}^{1 \times n}$ and $b \in \mathbb{R}$ are bias.

The hyperplane that is able to separate the positive and negative examples into two classifications based on one feature is called a separating hyperplane. The SVM algorithm finds the best among multiple Separating Hyperplanes, and we may determine the Large-margin Hyperplane by calculating whether the geometric margin between the nearest sample point and the separating plane is the largest. (Figure 4.3)

4.3 Kernel Function

Assuming that classification is not a linearly separable relationship, how do we split an already linearly inseparable data set? Here we will introduce the concept of the kernel function,

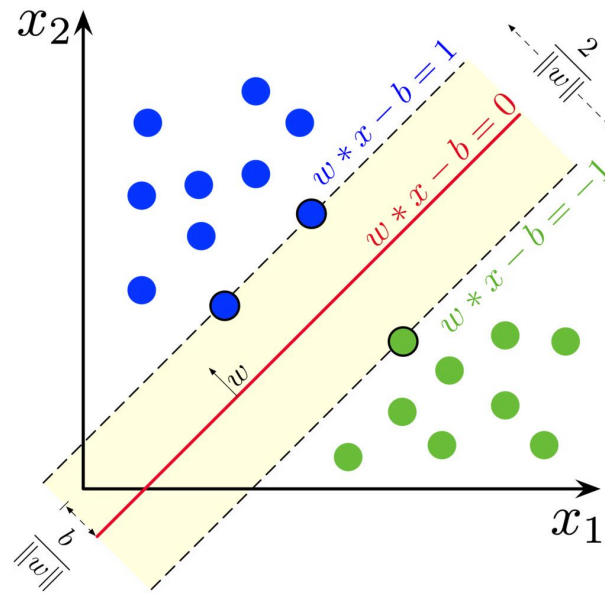


Figure 4.3: Separating Hyperplane [5]

which is a function that takes a vector in the original space as an input vector and returns the inner product of the vectors in the feature space. In this way, we use a mapping function to map samples from n -dimension to $n+1$ or higher dimensions (Figure 4.4).

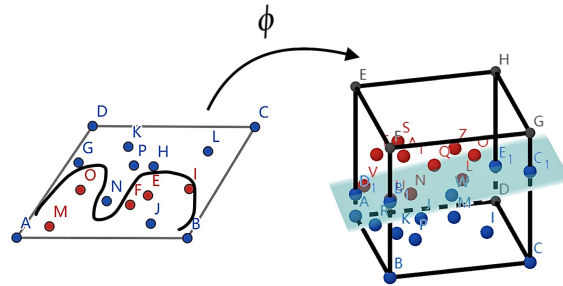


Figure 4.4: Transformation of Dimension

Hence, we can try to convert the original sample from linear inseparability in low dimensions to a linearly separable hyperplane in sufficiently high dimensions.

If there is $x_i, y_i \in X$ and a map $\phi(x) = X \rightarrow \mathbb{R}$,

$$k_{x_i, y_i} = \langle \phi_{x_i}, \phi_{y_i} \rangle = \phi(x_i) \cdot \phi(y_i) \quad (4.3)$$

This is to use the characteristics of the kernel function to construct a kernel function that can increase the dimension, which is just a convenient method for calculating the inner product after mapping to a high-dimensional space.

In the form of nested kernel functions, we can write any type of dataset's hyperplane.

$$b + \sum_{i=1}^n \alpha_i k(X_i, Y_i) \quad (4.4)$$

We have found through experiments that the types of kernel functions commonly used in SVM include:

$$k(X_i, Y_i) = \begin{cases} X_i^T Y_i \\ (rX_i^T Y_i + C)^d \\ \exp(-r|X_i - Y_i|^2) \end{cases} \quad (4.5)$$

From top to bottom, the 3 functions stand for **linear**, **polynomial**, and **RBF** functions, respectively (Equation 4.5).

4.4 Distinguishing Species # 5 Using FPNr (Task 1)

We find out that the best kernel function to split species #5 using FPNr is the **linear** kernel function. The result after 8-fold cross-validation is shown below:

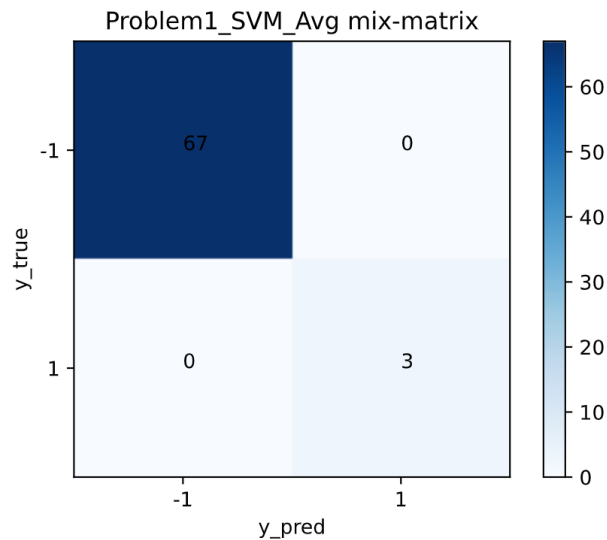


Figure 4.5: SVM Performance Matrix for Task 1

This is the performance matrix after averaging, in which 1 represents lizard species # 5, -1 represents other species, y_true stands for the actual species, and y_pred stands for the predicted species. The accuracy is **99.82%** and the mean square error (MSE) is 0.0071, which demonstrates great performance.

4.5 The Best Pair of Predicting Variables for Species # 5 (Task 2)

We identify that the best kernel function is the **polynomial** function. We also use a loop to iterate all the attributes, finding out that the best pair of predicting variables are medium body scales (**MBS**) and right side femoral pore number (**FRNr**). The Performance Matrix is demonstrated in Figure 4.6.

Where 1 represents lizard species # 5, -1 represents other species, y_true stands for the

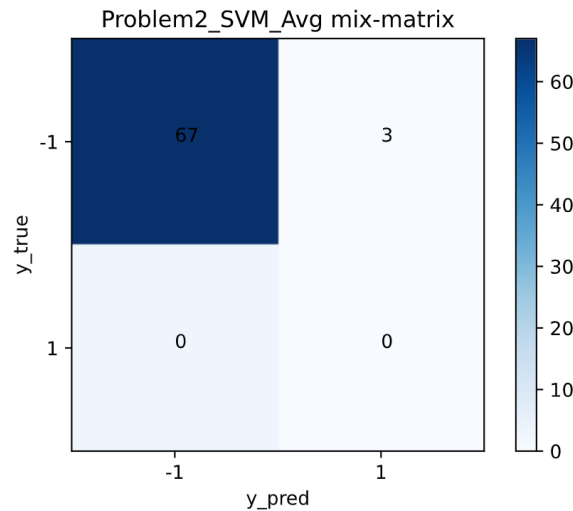


Figure 4.6: SVM Performance Matrix for Task 2

actual species, and y_pred stands for the predicted species. The overall accuracy is **95.74%** and MSE is 0.1984. We believe the decrease in accuracy is triggered by more uncertainty.

4.6 Distinguishing Sex (Task 3)

To accomplish the possibly highest accuracy, we set a loop to cover all the possible ways of the combination of attributes, which means the program will iterate $\sum_{i=1}^{23} C_i^{23} = 4116715363317$ times. Among all the possible combinations, the best accuracy occurs when 3 attributes are present, which are **VSN**, **HW**, and **PL**. The best kernel function is a **linear** function and the performance matrix is shown below:

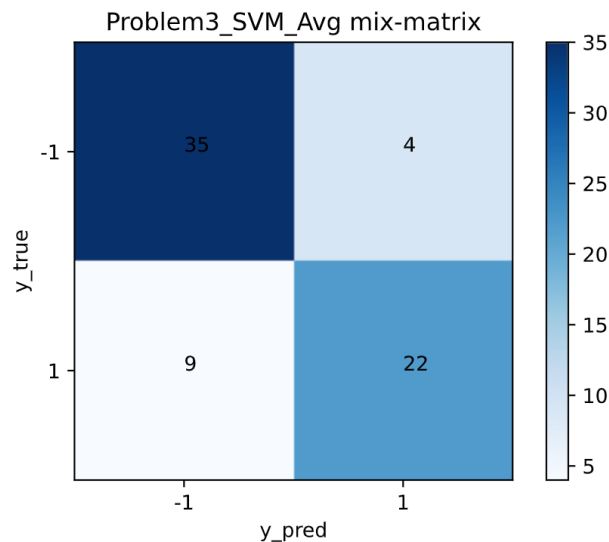


Figure 4.7: SVM Performance Matrix for Task 3

Where 1 represents a male lizard, -1 represents a female lizard, y_true is the actual species, and y_pred is the predicted species. The accuracy is **84.55%** and MSE is 0.1544.

4.7 Distinguishing Subgroup Species (Task 4)

4.7.1 Species # 6 and # 7 (4a)

Within all the possible combinations, we find the highest accuracy occurs when there is only one attribute presenting, which is **HFL**. The most suitable kernel function is **RBF** and the performance Matrix is shown below:

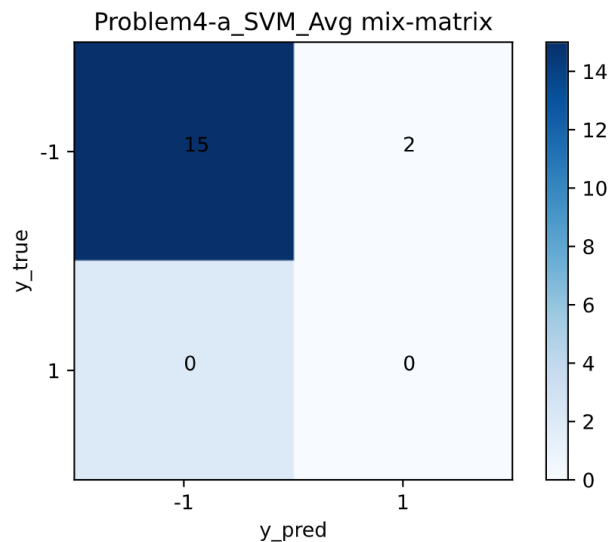


Figure 4.8: SVM Performance Matrix for Task 4a

Where 1 species # 7, -1 represents species # 6, y_true is the actual lizard's species, and y_pred is the predicted classification. The accuracy is **84.56%** and MSE is 0.154.

4.7.2 Species # 1 and # 2 (4b)

The highest accuracy occurs, when only one attribute is used, which **HFL**. The kernel function used is a **linear** function. The performance matrix is shown as follows:

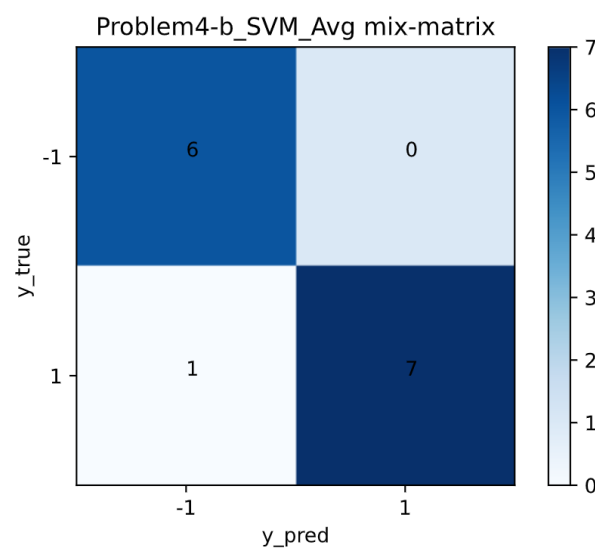


Figure 4.9: SVM Performance Matrix for Task 4b

Where 1 represents species # 2, -1 represents species # 1. y_{pred} is the predicted biological species and y_{true} is the actual lizards' species. The overall accuracy is **84.42%**, and MSE is 0.156.

4.7.3 Species # 3, # 4, and # 5 (4c)

We find when the model is trained with two attributes, the accuracy is the highest. The two attributes are **HFL** and **GSN** respectively. The most suitable kernel function is a **linear** function and the performance matrix is shown below:

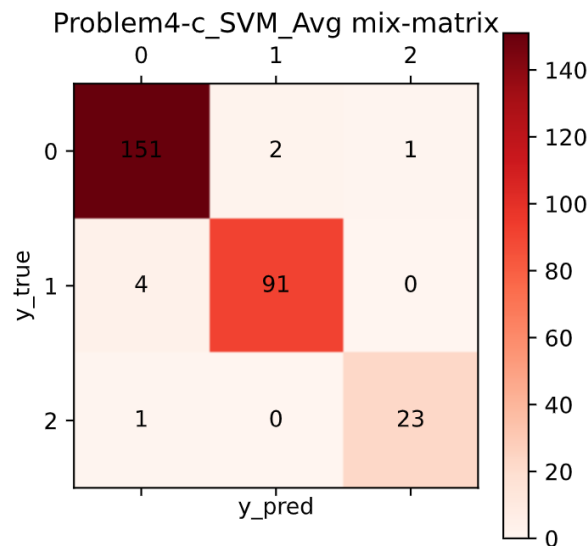


Figure 4.10: SVM Performance Matrix for Task 4c

In which 0 represents species # 3, 1 represents species # 4, and 2 represents species # 5. The true label stands for the actual species and the predicted label represents the predicted lizard species. The best accuracy is **89.00%** and the best MSE is 0.209.

4.8 Integrated Model- Predicting Species in the Entire Lizard Population (Task 5)

By integrating models, we build the criterion that is able to predict the lizards' species and sex. Here we present our ways of accomplishing it (Figure 4.11).

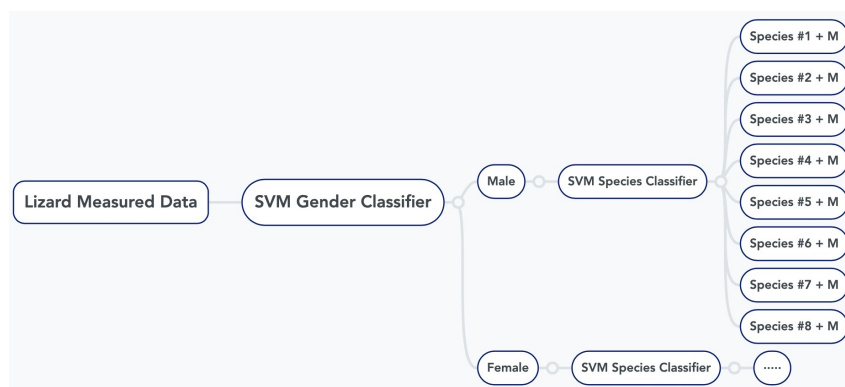


Figure 4.11: Flow Chart of the Integrated SVM Model

We basically divided the task into two processes. First, we distinguish the lizards' sex based on the SVM gender classifier (Mentioned in 4.6, Task 3). After gender is determined, we established another two SVM classifiers to differentiate biological species within a certain sex. The detailed results of SVM species classifiers are shown below:

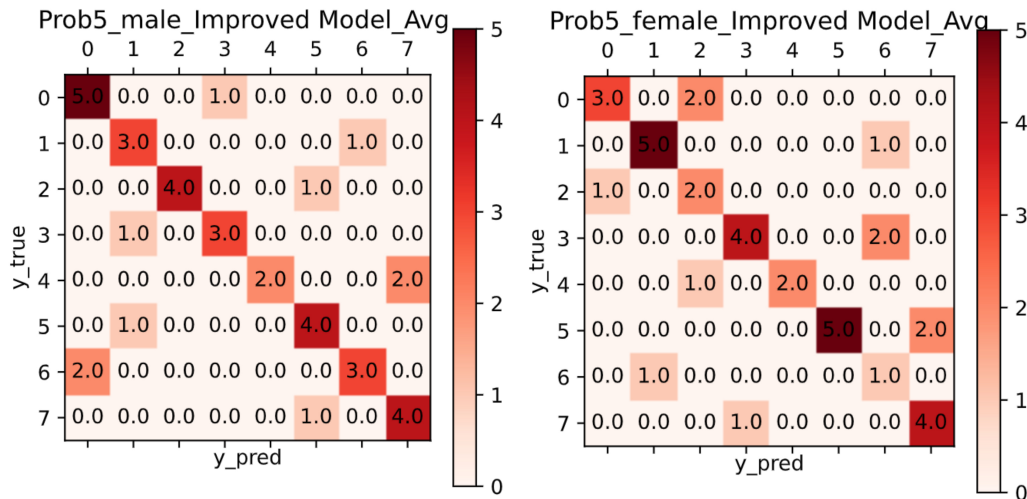


Figure 4.12: Left: Integrated Model for Male Lizards. Right: SVM Integrated Model for Female Lizards.

The accuracies of SVM species classifiers for males and females are 73.68% and 72.22%, respectively. After calculation, the overall accuracy of the integrated model that is able to distinguish both sex and biological species is **72.97%**, which is the averaged accuracy of male and female species classifier.

5 Alternative Models

Although the integrated SMV model is powerful, there could be ways to improve the accuracy given the specific problem. In this section, we will primarily focus on constructing models based on different theorems to achieve better accuracy for certain questions.

5.1 Adaptive Boosting Algorithm (Adaboost)

5.1.1 Basic Theorems

AdaBoost is an algorithm proposed by Yoav Freund and Robert Schapire in 1995 [6]. The AdaBoost algorithm is characterized by a high detection rate and is less prone to over-adaptation.

It is adaptive in the sense that samples wrongly classified by the previous basic classifier are strengthened, and the weighted whole sample is used again to train the next basic classifier. At the same time, a new weak classifier is added in each iteration until some predetermined small error rate is reached or a pre-specified maximum number of iterations is reached.

A typical AdaBoost structure is demonstrated as follows:

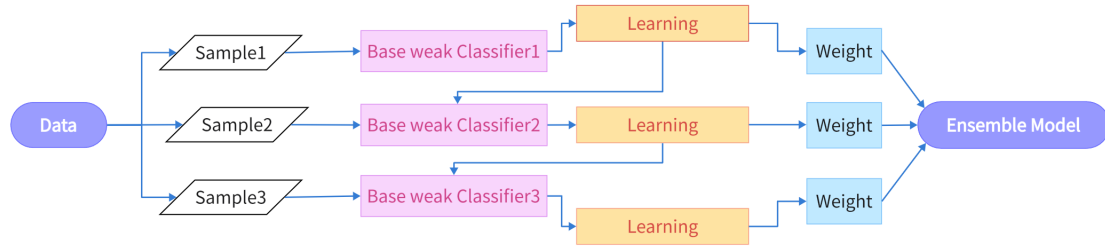


Figure 5.1: Structure of Adaboost Algorithm

5.1.2 Construction of Adaboost Algorithm

Suppose a training dataset $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$ is given (In this case, it is the measured lizard dataset), in which instance $x \in X$ and instance space $X \subset \mathbb{R}^n$

We construct the model as follows:

1. Initialize the weight distribution of the training samples. Each training sample is initially given the same weight value $D1(i)$

$$D1(i) = \frac{1}{m}, i = 1, 2, \dots, m \quad (5.1)$$

2. Perform multiple iterations with $m = 1, 2, \dots, M$ denotes the number of rounds of the iteration.
 - a. Learning using a training dataset with a weight distribution D_m to obtain the basic classifier $H_t : X \rightarrow \{-1, +1\}$. The error e_t of this weak classifier on the distribution D_t is:

$$e_t = P(H_t(x_i) \neq y_i) = \sum_{i=1}^N w_{ti} I(H_t(x_i) \neq y_i) \quad (5.2)$$

- b. Calculate the weight of this weak classifier in the final classifier a_t

$$a_t = \frac{1}{2} \ln\left(\frac{1 - e_t}{e_t}\right) \quad (5.3)$$

- c. Update the weight distribution D_{t+1} of the training samples:

$$D_{t+1} = \frac{D_t(i) \exp(-a_t y_i H_t(x_i))}{Z_t} \quad (5.4)$$

Where Z_t is a normalization factor, $Z_t = 2\sqrt{e_t(1 - e_t)}$

3. Combine each weak classifier according to each of their weight a_t , then a strong classifier H_{final} is obtained by the sign function:

$$H_{final} = \text{sign}\left(\sum_{t=1}^T a_t H_t(x)\right) \quad (5.5)$$

5.1.3 Task 2 Implementation

By conducting several attempts, we find out the optimal results will come out when the **number of weak classifiers = 100** and the **learning rate = 1**. The performance matrix is shown as follows:

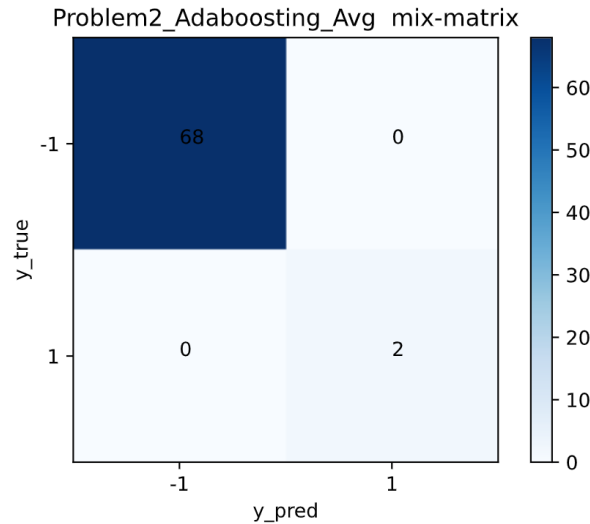


Figure 5.2: Adaboost Performance Matrix for Task 2

The best accuracy is shown to be **99.82%**, which is incredibly high. Compared with the former SMV classifier, the value of MSE also decreases and reaches a value of 0.007.

However, by our Adaboost algorithm, the best pair of prediction variables becomes **FPNr** and **HFL** (SVM classifier results in FPNr and MBS), which is considered to be more precise based on a higher accuracy in the testing data.

5.2 Random Forest

5.2.1 Basic Theorems

To improve the accuracy and optimize the algorithm, we will introduce another mathematical model based on the idea of Random Forest [7]. Random Forest (RF) is an algorithm that integrates multiple decision trees (the basic units of RF). Its essence belongs to a major branch of machine learning - the Ensemble Learning method.

RF is explainable and is able to handle numerical features. It is also unexcelled in its accuracy among all current algorithms. Due to the fact that an RF is composed of numerous decision trees, we will first introduce the construction of a single decision tree that is able to categorize lizards.

5.2.2 Decision Trees

We construct the decision trees based on Iterative Dichotomiser 3 (ID3) algorithm [8]. The core of ID3 algorithm is to select features at each node of the decision tree based on the principle of information gain (also called the Kullback–Leibler divergence), and to construct the decision tree recursively.

Information gain (IG) is the criterion we use and it refers to the difference in entropy (H)

before and after the set of data (X) has been spilt based on an attribute A , which is also equivalent to Information Entropy (H_X) - Conditional Entropy ($H_{X|Y}$) (Equation 5.6)

$$IG = H_X - H_{X|Y} \quad (5.6)$$

In which $H_{X|Y} = \sum_{z \in Z} P_z H_z$. Z is the subset created by splitting attribute Y from set X . P_z is the proportion of the number of elements in z to the number of elements in set X .

The value of IG tells us which attribute in a given set of training features is most useful for distinguishing. The attribute with the largest IG will be used first to spilt the original set X (in this case the measured lizard dataset). After that, we will continue spilt the data in accordance with IG until all the value of the remaining IG is small or there is no attribute left.

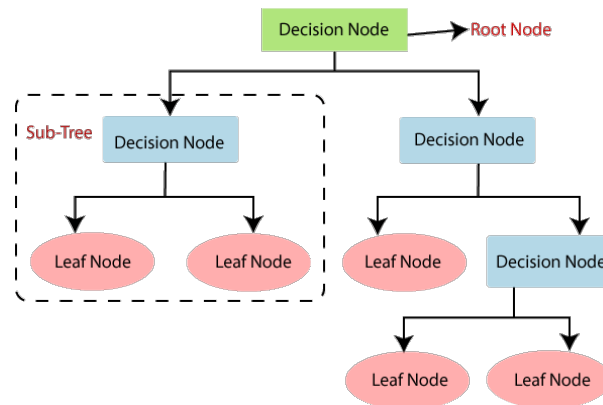


Figure 5.3: General Structure of a Decision Tree [9]

5.2.3 Construction of Random Forest

The result yielded by a single decision tree always fails to achieve the best accuracy, because a single decision tree is considered to be a weak classifier and the problem of overfitting will always appear. However, when numerous weak classifiers combine together, a strong classifier will be produced and the overall result will be the choice of the majority of decision trees.

Besides, the data we used to train each decision tree is relatively different. We will use the method called bootstrapping, which refers to random sampling with replacement, and such a method is suitable for datasets with a relatively small sample size.

This will result in a lower correlation between trees, leading to 99.9% uncorrelated trees making predictions that cover all cases. These results will further cancel each other out and eventually, a few decision trees will make a good prediction with great accuracy.

5.2.4 Task 3 Implementation

To construct the RF model that distinguishes male and female lizards, we first pre-process all the measured lizard data. Due to the large dataset and multiple attributes, we use Pipeline to achieve data normalization and we use Principal Component Analysis (PCA) to reduce the linear dimension.

Then we set the number of trees to 100 and use mean square error (MSE) as the criterion. First, we get the importance of each attribute based on IG (Figure 5.4). The ventral scale number on the middle line (VSN) occupies the greatest importance.

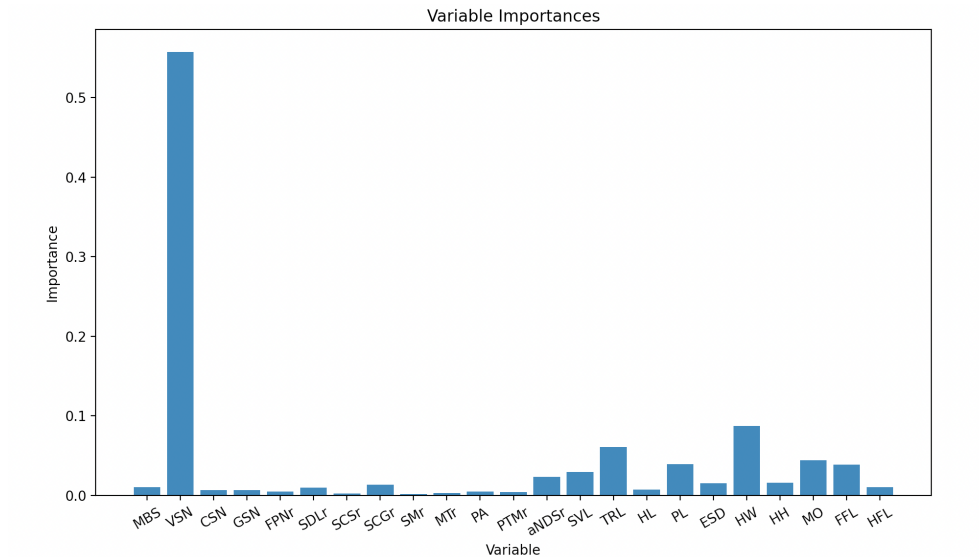


Figure 5.4: Information Gain of each attribute

We further get the structure of Random Forest utilizing RandomForestRegressor, in which X[1]-X[23] represent 23 attributes, from MBS to HFL (The detailed sequence is shown in Figure 5.2 X-axis). The results are shown as follows, and we can see that the MSE is gradually decreasing (Figure 5.5).

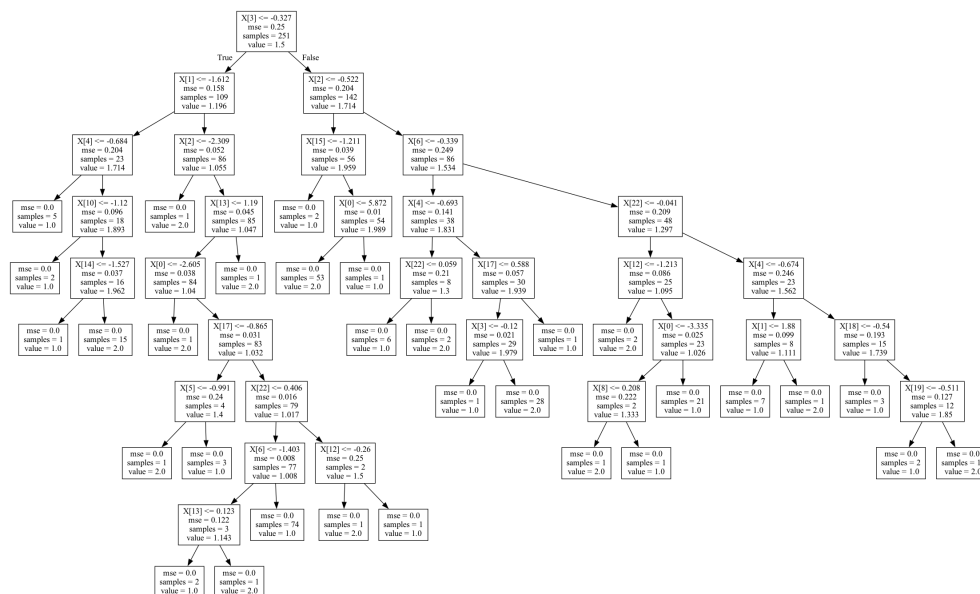


Figure 5.5: Structure of Random Forest

The accuracy is further evaluated by the performance matrix (Figure 5.6). There are 170 variables used to be testing data including 21 classification errors. The overall accuracy is **87.65 %**, which is higher than the results yielded by SVM Model and displays robustness and precision.

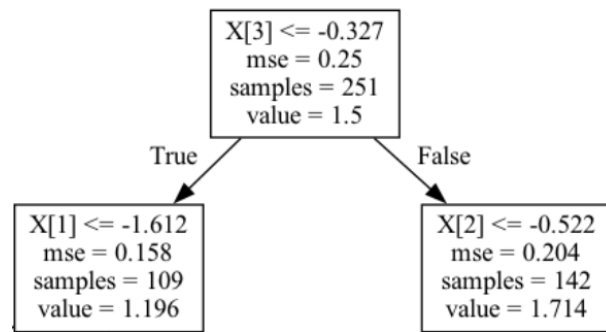


Figure 5.6: Detailed Structure of Random Forest

Here we also provide a detailed structure of the random forest, in which the first line represents the classification condition, mse stands for mean square error, and samples represents the number of samples that needs to be classified on each node. Because the output of Random Forest is numerical values 1 or 2, value stands for the averaged outputting value.

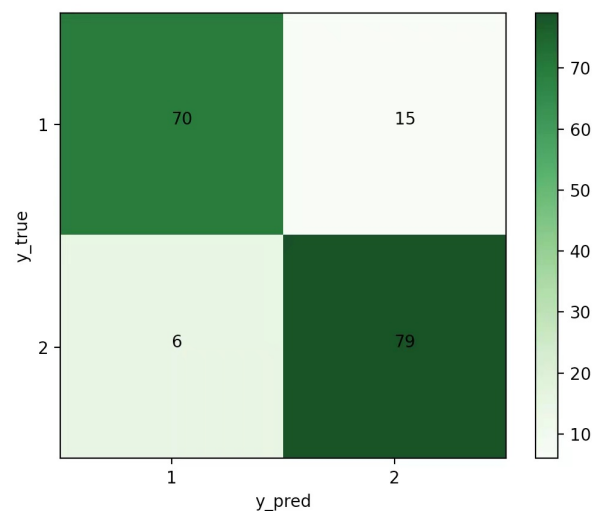


Figure 5.7: Performance Matrix

In the performance matrix of Random Forest, 1 refers to the male lizard, 2 refers to the female lizard, y_true stands for the actual measured gender, and y_pred stands for the predicted classification.

5.2.5 Improved Random Forest Model

Again back to Figure 5.4, there are still some attributes with insignificant importance existing. We will use RandomForestRegressor again on the condition that all trivial attributes are eliminated. We expected all the noises that affect the accuracy will be erased and the precision of the model could be enhanced. Here are the final results.

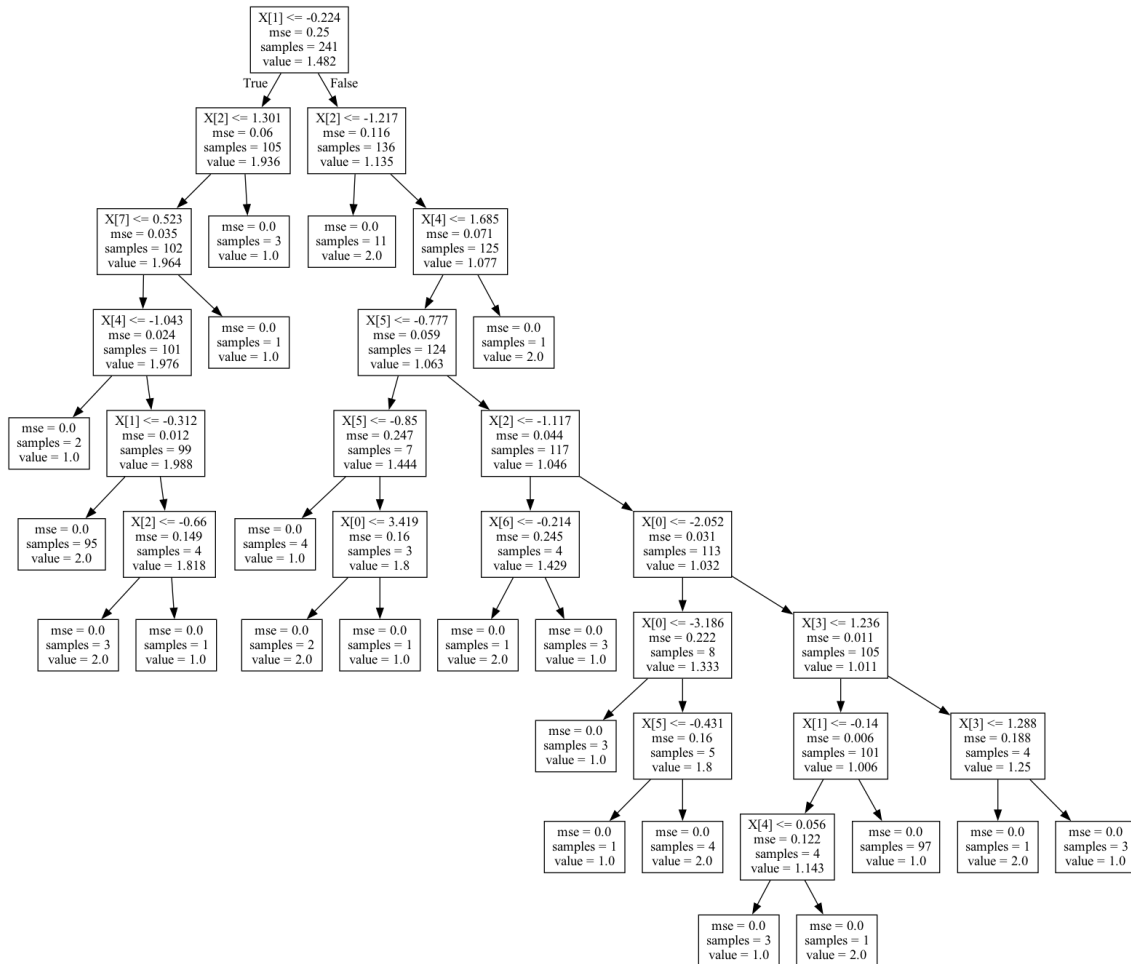


Figure 5.8: Improved Random Forest

Based on Figure 5.9 shown in the next page, there are only 19 classification errors (21 in the original model), and the rest 151 sex numbers have been predicted correctly (149 in the original model). The final accuracy of the RF model is **88.82 %**, which successfully increases by 1.17%, and that demonstrates that unrelated elements such as noises have been eliminated and the model is being improved. It also demonstrates that our theoretical foundation is robust.

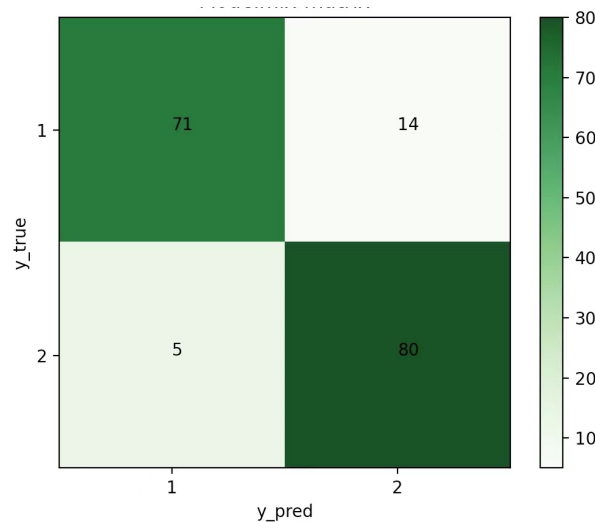


Figure 5.9: Improved Performance Matrix

5.3 Improved Integrated Model

5.3.1 Basic Theroem

Recall that for task 5, the integrated model (mentioned in 4.8) combines two SVM models, in which one is used to distinguish sex and the other is used to differentiate species. With the improved sex classifier with higher accuracy (Random Forest), the overall accuracy of the integrated model used in solving task 5 could also get enhanced. Due to this reason, we substitute SVM gender classifier with Random Forest gender classifier. Here is the improved structure of the integrated model:



Figure 5.10: Improved Structure of the Integrated Model

5.3.2 Task 5 Implementation

We generate the performance matrix of the improved integrated model as shown in Figure 5.11.

In which 0-7 represents lizard species # 1 - # 8 respectively. The accuracies of improved integrated model for males and females are 80.56% and 78.38%, respectively. After calculating the averaged accuracies of two models, the overall accuracy of the improved integrated model

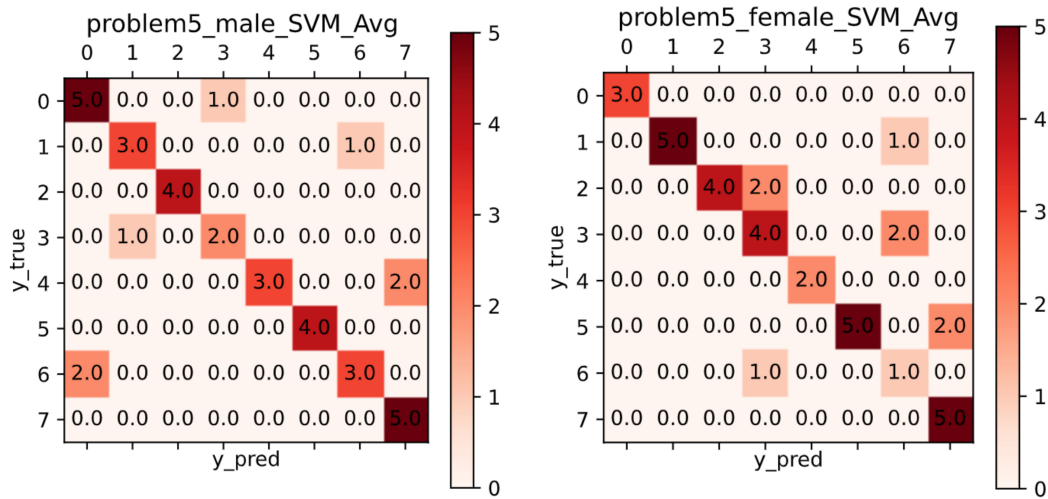


Figure 5.11: Left: Improved Integrated Model for Male Lizards. Right: Improved Integrated Model for Female Lizards

that is able to distinguish both sex and biological species improves by 7.25% and reaches **79.47%** (Originally 72.22%).

6 Sensitivity Analysis

6.1 SMV

For the sensitivity analysis of SMV, we change the type of kernel function and compare the final accuracy and MSE. We typically use 3 kernel functions: Linear, Polynomial, and RBF.

Accuracy and MSE of Different Kernel Functionl			
Number of Task	Linear	Polynomial	RBF
Task 1	99.82% / 0.007	99.64% / 0.014	99.64% / 0.014
Task 2	95.32% / 0.137	95.74% / 0.017	95.04% / 0.193
Task 3	81.43% / 0.164	80.00% / 0.165	78.57% / 0.172
Task 4a	84.56% / 0.154	82.35% / 0.168	82.35% / 0.168
Task 4b	84.42% / 0.156	78.21% / 0.217	76.61% / 0.233
Task 4c	89.00% / 0.209	85.35% / 0.234	88.26% / 0.227

6.2 Adaboost

For the sensitivity analysis of Adaboost algorithm, we change two parameters: the number of weak classifiers and learning rate. The outcome is shown as follows:

Number of Weak Classifiers	MSE	Accuracy
10	0.0212	99.46%
50	0.0212	99.46%
100	0.00714	99.82% %
200	0.0141	99.64 %
Learning Rate	MSE	Accuracy
1	0.00714	99.82%
0.1	0.0141	99.64%
0.01	0.0212	99.46%

The results change negligibly when the parameter changes. However, there is still an optimal situation in which the number of weak classifiers is 100 and the learning rate is 1. Such a situation also meets our expectation that more classifiers enable a more comprehensive weight matrix and a small learning rate corresponds to data with a relatively small variance.

6.3 Random Forest

For the sensitivity analysis of Random Forest, we also change two parameters: the number of decision trees and bootstrapping. The final accuracy and MSE are shown in the following two tables:

Number of Trees	MSE (train)	MSE (test)	Accuracy
1000	0.008	0.085	88.23%
100	0.008	0.085	88.23%
50	0.009	0.086	88.23%
10	0.013	0.091	86.47%

This demonstrates Random Forest reaches its best performance when the number of trees is 100. Besides, when the number of trees exceeds 50, we believe it is enough to extract features that have covered all the features of the training data, so the accuracy rate could fluctuate less and the model could be more robust.

Bootstrapping	MSE (train)	MSE (test)	Accuracy
ON	0.008	0.085	88.23%
OFF	0.000	0.138	82.30%

This demonstrates that sampling based on the theorem of Bootstrapping will increase the overall accuracy because it can undermine the biases, which meets our expectations.

7 Strengths and Weaknesses

7.1 Strengths

- **Integrated Model**

We have built SVM models that can tackle all questions about lizard differentiation based on the same core theorems- SVM. It is very convenient and it superbly demonstrates how solid our theories and models are.

- **Specialized Model**

There could also be models with better accuracy in certain scenarios, so we have made specialized models that perfectly fit that kinds of circumstances. Therefore, our criteria fit all situations and can give precise and comprehensive results when biologists conduct field research.

7.2 Weaknesses

- **Limitations of Data**

We do not have perfectly sufficient data to train our model. We have used 70% of the measured data as training data and 30% as our test data. However, because the number of data is limited, which means if there are more data for training, the model would produce more accurate outcomes.

8 Conclusion and Future Work

Considering all, we have established three models to differentiate the lizards' species and they work effectively and magnificently as we expected. The general model based on SVM classifier works well for most situations, and we also have specialized models targeted at certain tasks that produce satisfying outcomes, which perfectly fit every situation as programmed.

There are still many puzzles in other fields besides lizards differentiation and they are consequential as well such as salmon differentiation, beetles categorization, and lots of things like them. The model will be utilized in other fields as we are looking for. In the future, we will keep evaluating this model to make it more and more accurate which helps biologists do their work in the field and that's our purpose in doing it.

References

- [1] Precise paternal ancestry of hybrid unisexual ZW lizards (genus *Darevskia*: Lacertidae: Squamata) revealed by Z-linked genomic markers
- [2] New subspecies of meadow lizard, *Darevskia praticola loriensis* ssp. nov. (Reptilia: Sauria) from Armenia.
- [3] Beolens, Bo; Watkins, Michael; Grayson, Michael (2011). The Eponym Dictionary of Reptiles. Baltimore: Johns Hopkins University Press. xiii + 296 pp. ISBN 978-1-4214-0135-5. (*Darevskia*, p. 65).
- [4] https://www.researchgate.net/figure/Diagram-of-k-fold-cross-validation-with-k-10-Image-from-Karl-Rosaen-Log_fig1_332370436
- [5] https://en.m.wikipedia.org/wiki/Support_vector_machine
- [6] Robert E. Schapire AT&T Labs Research Shannon Laboratory. The Boosting Approach to Machine Learning An Overview
- [7] Biau, G., Scornet, E. A random forest guided tour. TEST 25, 197–227 (2016). <https://doi.org/10.1007/s11749-016-0481-7>
- [8] Wei Peng, Juhua Chen, and Haiping Zhou. An Implementation of ID3 – Decision Tree Learning Algorithm. Retrieved March 10, 2012, from <http://web.arch.usyd.edu.au/wpeng/DecisionTree2.pdf>.
- [9] <https://www.javatpoint.com/machine-learning-decision-tree-classification-algorithm>

Appendices

Appendix A Task 4c. SVM Multiclass Classification

```
#!/usr/bin/env python
# coding: utf-8

# In[1]:

import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
import numpy as np
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, mean_squared_error
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt

# In[2]:
```

```
df = pd.read_excel(r"C:\Users\Desktop\IMMC23 Prob_E data.xlsx")
```

```
# In[3]:
```

```
mapper = {}
for i in range(1,9):
    if i == 3 :
        mapper[i] = 1
    elif i == 4:
        mapper[i] = 2
    elif i ==5:
        mapper[i] = 3
    else:
        mapper[i] = -1

df["target"] = df["Species_num"].map(mapper)
df = df.loc[df['target']>0]
```

```
# In[4]:
```

```
features3 = ["MBS", "VSN", "CSN", "GSN", "FPNr", "SDLr", "SCSr", "SCGr", "SMr", "MTr", "PA", "PTMr",
             "aANDSr", "SVL", "TRL", "HL", "PL", "ESD", "HW", "HH", "MO", "FFL", "HFL"]
best_score = 0
confusion = []
for x in range(len(features3)):
    for z in range(len(features3)):
        X = df[[features3[z], features3[x]]]
        Y = df["target"]
        kf = StratifiedKFold(n_splits=8)
        kfold = kf.split(X, Y)
        M11 = 0
        M12 = 0
        M13 = 0
        M21 = 0
        M22 = 0
        M23 = 0
        M31 = 0
        M32 = 0
        M33 = 0
        acc = []
        mse = []
        for k, (train, test) in enumerate(kfold):

            ss = StandardScaler()
            train_X = ss.fit_transform(X.iloc[train].values.reshape(-1,2))
            test_X = ss.fit_transform(X.iloc[test].values.reshape(-1,2))
            train_Y = Y.iloc[train].values
            test_Y = Y.iloc[test].values

            model = SVC(kernel='rbf')
            model.fit(train_X, train_Y)

            predictions = model.predict(test_X)
```



```

    for i in range(len(predictions)):
        if (predictions[i] == 1 and test_Y[i] == 1):
            M11 += 1
        elif (predictions[i] == 1 and test_Y[i] == 2):
            M12 += 1
        elif (predictions[i] == 1 and test_Y[i] == 3):
            M13 += 1
        elif (predictions[i] == 2 and test_Y[i] == 1):
            M21 += 1
        elif (predictions[i] == 2 and test_Y[i] == 2):
            M22 += 1
        elif (predictions[i] == 2 and test_Y[i] == 3):
            M23 += 1
        elif (predictions[i] == 3 and test_Y[i] == 1):
            M31 += 1
        elif (predictions[i] == 3 and test_Y[i] == 2):
            M32 += 1
        elif (predictions[i] == 3 and test_Y[i] == 3):
            M33 += 1
    acc.append(accuracy_score(test_Y,predictions))
    mse.append(mean_squared_error(test_Y,predictions))
#    C = confusion_matrix(test_Y,predictions,labels=['3','4','5'])
    if((sum(acc))/8) > best_score:
        best_score = (sum(acc))/8
        best_mse = (sum(mse))/8
        best_feature = (features3[z],features3[x])
        best_M11 = M11
        best_M12 = M12
        best_M13 = M13
        best_M21 = M21
        best_M22 = M22
        best_M23 = M23
        best_M31 = M31
        best_M32 = M32
        best_M33 = M33

# plot_fig(confusion_sum, "SVM_Avg ")
print("best acc",str((sum(acc))/8))
print("best mse",str((sum(mse))/8))
print(f"best feature pair {best_feature}")
print("Note: we record 3 as 1 and 4 as 2 and 5 as 3")

# In[5]:

M = [[best_M11,best_M12,best_M13],[best_M21,best_M22,best_M23],[best_M31,best_M32,best_M33]]
M = np.array(M)
M

# In[6]:

plt.matshow(M, cmap=plt.cm.Reds)
# plt.colorbar()
name_title = 'Problem4-c_SVM_Avg mix-matrix'
for i in range(len(M)):

```

```
    for j in range(len(M)):
        plt.annotate(M[j, i], xy=(i, j), horizontalalignment='center', verticalalignment='top')

# plt.tick_params(labelsize=15)

plt.ylabel('y_true')
plt.xlabel('y_pred')
plt.title(name_title)
plt.colorbar()
plt.savefig(name_title+"Prob4-confusion_matrix.svg")
plt.show()

# In[ ]:
```

Appendix B Task 2. Adaboost Algorithm

```
#!/usr/bin/env python
# coding: utf-8

# In[63]:

from sklearn.ensemble import AdaBoostClassifier
import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
import numpy as np
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, mean_squared_error
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt

# In[64]:

df = pd.read_excel(r"C:\Users\Desktop\IMMC23 Prob_E data.xlsx")

# In[66]:

mapper = {}
for i in range(1, 9):
    if i == 5:
        mapper[i] = 1
    else:
        mapper[i] = -1

df["Species_num"] = df["Species_num"].map(mapper)

# In[69]:
```

Appendix C Task 3. SVM Classifier

```
#!/usr/bin/env python
# coding: utf-8

# In[2]:

import pandas as pd
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler #
import numpy as np
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, mean_squared_error
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt

# In[3]:

def plot_fig(confusion, name_title):
    print(name_title + "The confusion matrix is ")
    print(confusion)
    plt.figure()
    classes = ['-1', '1']
    plt.imshow(confusion, cmap=plt.cm.Blues)
    indices = range(len(confusion))
    plt.xticks(indices, classes)
    plt.yticks(indices, classes)
    plt.colorbar()
    plt.xlabel('y_pred')
    plt.ylabel('y_true')
    plt.title(name_title + "mix-matrix")

    for first_index in range(len(confusion)):
        for second_index in range(len(confusion[first_index])):
            plt.text(first_index, second_index, confusion[first_index][second_index])

    plt.savefig(name_title + "Prob3SVM.svg")
    plt.show()

# In[4]:

df = pd.read_excel(r"C:\Users\Desktop\IMMC23 Prob_E data.xlsx")

# In[5]:

mapper = {}
for i in range(1, 3):
    if i == 1:
        mapper[i] = 1
    else:
```

```
mapper[i] = -1

df["Sex_num"] = df["Sex_num"].map(mapper)

# In[6]:

features3 = ["aNDsr", "SVL", "TRL", "HL", "PL", "ESD", "HW", "HH", "MO", "FFL", "HFL"]
best_score = 0
for x in range(len(features3)):
    for z in range(len(features3)):
        for j in range(len(features3)):
            X = df[[features3[z], features3[x], features3[j]]]
            Y = df["Sex_num"]
            kf = StratifiedKFold(n_splits=8)
            kfold = kf.split(X, Y)
            TP = 0
            FP = 0
            TN = 0
            FN = 0
            acc = []
            mse = []
            confusion = []
            for k, (train, test) in enumerate(kfold):

                ss = StandardScaler()
                train_X = ss.fit_transform(X.iloc[train].values.reshape(-1, 3))
                test_X = ss.fit_transform(X.iloc[test].values.reshape(-1, 3))
                train_Y = Y.iloc[train].values
                test_Y = Y.iloc[test].values

                model = SVC(kernel='linear')
                model.fit(train_X, train_Y)

                predictions = model.predict(test_X)
                for i in range(len(predictions)):
                    if predictions[i] == 1 and test_Y[i] == 1:
                        TP += 1
                    elif predictions[i] == 1 and test_Y[i] == -1:
                        FP += 1
                    elif predictions[i] == -1 and test_Y[i] == -1:
                        TN += 1
                    elif predictions[i] == -1 and test_Y[i] == 1:
                        FN += 1
                acc.append(accuracy_score(test_Y, predictions))
                mse.append(mean_squared_error(test_Y, predictions))

            # print('K=', k)
            # print('TP=', TP)
            # print('FP=', FP)
            # print('TN=', TN)
            # print('FN=', FN)

        if ((sum(acc))/8) > best_score:
            best_score = (sum(acc))/8
            best_mse = (sum(mse))/8
```

```

        best_feature = (features3[z], features3[x], features3[j])
        best_TP = TP
        best_FP = FP
        best_TN = TN
        best_FN = FN
        confusion.append(confusion_matrix(test_Y, predictions))
        confusion_sum = sum(confusion)//len(confusion)

plot_fig(confusion_sum, "Prob3_SVM_Avg ")
print("best acc", str((sum(acc))/8))
print("best mse", str((sum(mse))/8))
print(f"best feature pair {best_feature}")
print("Note: we record male as 1 and female as 2")

# In[ ]:

```

Appendix D Task 3. Random Forest

```

import sklearn
from sklearn.ensemble import RandomForestRegressor
from sklearn.datasets import make_regression

from sklearn.ensemble import RandomForestRegressor
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from sklearn.metrics import r2_score
from sklearn.metrics import mean_squared_error

df = pd.read_csv('IMMC_REVISIED.txt', sep='\s+')

print(df.head())

X = df.iloc[:, :-1].values

y = df.iloc[:, -1].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1)

forest = sklearn.ensemble.RandomForestRegressor(
    n_estimators=10,
    criterion='mse',
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    min_weight_fraction_leaf=0.0,
    max_features='auto',
    max_leaf_nodes=None,
    min_impurity_decrease=0.0,

```

```
#min_impurity_split=None,
bootstrap=True,
oob_score=False,
n_jobs=None,
random_state=None,
verbose=0,
warm_start=False,
ccp_alpha=0.0,
max_samples=None)

forest.fit(X_train, y_train)
y_train_pred = forest.predict(X_train)
y_test_pred = forest.predict(X_test)

print('MSE train: %.3f, test: %.3f' % (
    mean_squared_error(y_train, y_train_pred),
    mean_squared_error(y_test, y_test_pred)))
print('R^2 train: %.3f, test: %.3f' % (
    r2_score(y_train, y_train_pred),
    r2_score(y_test, y_test_pred)))

# Get numerical feature importances
importances = list(forest.feature_importances_)
print(importances)

# Saving feature names for later use
feature_list = list(df.columns)[0:8]

feature_importances = [(feature, round(importance, 3)) for feature, importance in zip(feature_list, importances)]
# Sort the feature importances by most important first
feature_importances = sorted(feature_importances, key = lambda x: x[1], reverse = True)

# Print out the feature and importances

print(feature_importances)

import matplotlib.pyplot as plt
# Set the style
# plt.style.use('fivethirtyeight')
# list of x locations for plotting
x_values = list(range(len(importances)))
print(x_values)
# Make a bar chart
plt.bar(x_values, importances, orientation = 'vertical')
# Tick labels for x axis
plt.xticks(x_values, feature_list, rotation=30)
# Axis labels and title
plt.ylabel('Importance'); plt.xlabel('Variable'); plt.title('Variable Importances');
plt.show()

from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

pipe = Pipeline([('scaler', StandardScaler()), ('reduce_dim', PCA()),
                ('regressor', forest)])
pipe.fit(X_train, y_train)
ypipe = pipe.predict(X_test)
```

```
from six import StringIO
from IPython.display import Image
from sklearn.tree import export_graphviz
import pydotplus
import os

dot_data = StringIO()
export_graphviz(pipe.named_steps['regressor'].estimators_[0],
                out_file=dot_data)
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
graph.write_png('tree.png')
Image(graph.create_png())

def plot_fig(confusion,name_title):
    print(name_title+"Matrix is ")
    print(confusion)
    plt.figure()
    classes=['1','2']
    plt.imshow(confusion, cmap=plt.cm.Greens)
    indices = range(len(confusion))
    plt.xticks(indices, classes)
    plt.yticks(indices, classes)
    plt.colorbar()
    plt.xlabel('y_pred')
    plt.ylabel('y_true')
    plt.title(name_title+"mix-matrix")

    for first_index in range(len(confusion)):
        for second_index in range(len(confusion[first_index])):
            plt.text(first_index, second_index, confusion[first_index][second_index])

    plt.savefig(name_title+"The Performance Matrix is.jpg")
    plt.show()

y_test_pred[y_test_pred<1.5]=1
y_test_pred[y_test_pred>=1.5]=2
from sklearn.metrics import accuracy_score

print("ACC={}".format(accuracy_score(y_test, y_test_pred)))

from sklearn.metrics import confusion_matrix
confusion=confusion_matrix(y_test, y_test_pred)
plot_fig(confusion, 'Model')
print("Length of testing data:",len(y_test))
index = np.arange(0,len(y_test))
true_lis = index[y_test == y_test_pred]
false_lis = index[y_test != y_test_pred]
print('Elements that are predicted correctly :\n',true_lis)
print('Elements that are predicted incorrectly :\n',false_lis)
#-----#
```
